

Presentación:

Nombre:

Algelis Severino

Tema:

Introducción a la Programación y Diagrama de flujo.

Tecnico:

Ciberseguridad Semipresencial

Facilitador:

Kevin Feliz Henriquez

Centro:

Diapiato Pro salud

Descripción:

En este documento .pdf adjunto capturas del reto 11 de 30 days of code de HackerRank.

Link GitHub:

<https://github.com/SeverinoCyber/30-days-of-code/tree/main/Hackerrank/day17>

Python3:

Submissions

Leaderboard

Discussions

Editorial

Tutorial

HackerRank

Prepare > Tutorials > 30 Days of Code > Day 17: More Exceptions

Exit Full Screen View

Objective

Yesterday's challenge taught you to manage exceptional situations by using try and catch blocks. In today's challenge, you will practice throwing and propagating an exception. Check out the Tutorial tab for learning materials and an instructional video.

Task

Write a Calculator class with a single method: `int power(int,int)`. The power method takes two integers, `n` and `p`, as parameters and returns the integer result of n^p . If either `n` or `p` is negative, then the method must throw an exception with the message: `n and p should be non-negative`.

Note: Do not use an access modifier (e.g.: `public`) in the declaration for your Calculator class.

Input Format

Input from stdin is handled for you by the locked stub code in your editor. The first line contains an integer, `T`, the number of test cases. Each of the `T` subsequent lines describes a test case in 2 space-separated integers that denote `n` and `p`, respectively.

Constraints

- No Test Case will result in overflow for correctly written code.

Output Format

Output to stdout is handled for you by the locked stub code in your editor. There are `T` lines of output, where each line contains the result of n^p as calculated by your Calculator class' power method.

Sample Input

```
4
3 5
2 4
-1 -2
-1 3
```

Sample Output

```
243
16
n and p should be non-negative
n and p should be non-negative
```

Change Theme

Language

Python 3

↕

⋮

```
1 #Write your code here
2 class Calculator:
3     def power(self, n, p):
4         if n < 0 or p < 0:
5             raise Exception("n and p should be non-negative")
6         return n**p
7
8
9
10 myCalculator=Calculator()
11 T=int(input())
12 for i in range(T):
13     n,p = map(int, input().split())
14     try:
15         ans=myCalculator.power(n,p)
16         print(ans)
17     except Exception as e:
18         print(e)
```

Line: 18 Col: 28

⬇ Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Submissions

Leaderboard

Discussions

Editorial

Tutorial

HackerRank

Prepare > Tutorials > 30 Days of Code > Day 17: More Exceptions

Exit Full Screen View

Objective

Yesterday's challenge taught you to manage exceptional situations by using try and catch blocks. In today's challenge, you will practice throwing and propagating an exception. Check out the Tutorial tab for learning materials and an instructional video.

Task

Write a Calculator class with a single method: `int power(int,int)`. The power method takes two integers, `n` and `p`, as parameters and returns the integer result of n^p . If either `n` or `p` is negative, then the method must throw an exception with the message: `n and p should be non-negative`.

Note: Do not use an access modifier (e.g.: `public`) in the declaration for your Calculator class.

Input Format

Input from stdin is handled for you by the locked stub code in your editor. The first line contains an integer, `T`, the number of test cases. Each of the `T` subsequent lines describes a test case in 2 space-separated integers that denote `n` and `p`, respectively.

Constraints

- No Test Case will result in overflow for correctly written code.

Output Format

Output to stdout is handled for you by the locked stub code in your editor. There are `T` lines of output, where each line contains the result of n^p as calculated by your Calculator class' power method.

Sample Input

```
4
3 5
2 4
-1 -2
-1 3
```

Sample Output

```
243
16
n and p should be non-negative
n and p should be non-negative
```

Line: 18 Col: 28

⬇ Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

Next Challenge

Test case 0

Test case 1

Input (stdin)

Download

```
1 4
2 3 5
3 2 4
4 -1 -2
5 -1 3
```

Expected Output

Download

```
1 243
2 16
3 n and p should be non-negative
4 n and p should be non-negative
```

JavaScripts:

Submissions

Leaderboard

Discussions

Editorial

Tutorial

HackerRank

Prepare > Tutorials > 30 Days of Code > Day 17: More Exceptions

Objective

Yesterday's challenge taught you to manage exceptional situations by using try and catch blocks. In today's challenge, you will practice throwing and propagating an exception. Check out the [Tutorial](#) tab for learning materials and an instructional video.

Task

Write a Calculator class with a single method: `int power(int,int)`. The power method takes two integers, `n` and `p`, as parameters and returns the integer result of n^p . If either `n` or `p` is negative, then the method must throw an exception with the message: `n and p should be non-negative`.

Note: Do not use an access modifier (e.g.: `public`) in the declaration for your Calculator class.

Input Format

Input from stdin is handled for you by the locked stub code in your editor. The first line contains an integer, `T`, the number of test cases. Each of the `T` subsequent lines describes a test case in 2 space-separated integers that denote `n` and `p`, respectively.

Constraints

- No Test Case will result in overflow for correctly written code.

Output Format

Output to stdout is handled for you by the locked stub code in your editor. There are `T` lines of output, where each line contains the result of n^p as calculated by your Calculator class' power method.

Sample Input

```
4
3 5
2 4
-1 -2
-1 3
```

Sample Output

```
243
16
```

Change Theme

Language

JavaScript (Node.js)

⌵

⌵

```
8 process.stdin.on('data', function (data) {
9     //ignore this code
10 });
11
12 process.stdin.on('end', function () {
13     input_stdin_array = input_stdin.split("\n");
14     main();
15 });
16 function readLine() {
17     return input_stdin_array[input_currentline++];
18 }
19
20 //Write your code here
21 class Calculator{
22     power(n,p){
23         if(n < 0 || p < 0){
24             throw "n and p should be non-negative";
25         }
26         return n**p;
27     }
28 }
29 function main(){
30     var myCalculator=new Calculator();
31     var T=parseInt(readLine());
32     while(T-->0){
33         var num = (readLine().split(" "));
34         try{
35             var n=parseInt(num[0]);
36             var p=parseInt(num[1]);
37             var ans=myCalculator.power(n,p);
38             console.log(ans);
39         }
40         catch(e){
41             console.log(e);
42         }
43     }
44 }
```

Line: 29 Col: 17

⬇ Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Submissions

Leaderboard

Discussions

Editorial

Tutorial

HackerRank

Prepare > Tutorials > 30 Days of Code > Day 17: More Exceptions

Objective

Yesterday's challenge taught you to manage exceptional situations by using try and catch blocks. In today's challenge, you will practice throwing and propagating an exception. Check out the [Tutorial](#) tab for learning materials and an instructional video.

Task

Write a Calculator class with a single method: `int power(int,int)`. The power method takes two integers, `n` and `p`, as parameters and returns the integer result of n^p . If either `n` or `p` is negative, then the method must throw an exception with the message: `n and p should be non-negative`.

Note: Do not use an access modifier (e.g.: `public`) in the declaration for your Calculator class.

Input Format

Input from stdin is handled for you by the locked stub code in your editor. The first line contains an integer, `T`, the number of test cases. Each of the `T` subsequent lines describes a test case in 2 space-separated integers that denote `n` and `p`, respectively.

Constraints

- No Test Case will result in overflow for correctly written code.

Output Format

Output to stdout is handled for you by the locked stub code in your editor. There are `T` lines of output, where each line contains the result of n^p as calculated by your Calculator class' power method.

Sample Input

```
4
3 5
2 4
-1 -2
-1 3
```

Sample Output

```
243
16
```

⬇ Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

Next Challenge

✔ Test case 0

✔ Test case 1

Input (stdin)

Download

```
1 4
2 3 5
3 2 4
4 -1 -2
5 -1 3
```

Expected Output

Download

```
1 243
2 16
3 n and p should be non-negative
4 n and p should be non-negative
```

C#:

Submissions

Leaderboard

Discussions

Editorial

Tutorial

HackerRank

Prepare > Tutorials > 30 Days of Code > Day 17: More Exceptions

Objective

Yesterday's challenge taught you to manage exceptional situations by using try and catch blocks. In today's challenge, you will practice throwing and propagating an exception. Check out the [Tutorial](#) tab for learning materials and an instructional video.

Task

Write a Calculator class with a single method: `int power(int n, int p)`. The power method takes two integers, `n` and `p`, as parameters and returns the integer result of n^p . If either `n` or `p` is negative, then the method must throw an exception with the message: `n and p should be non-negative`.

Note: Do not use an access modifier (e.g.: `public`) in the declaration for your Calculator class.

Input Format

Input from stdin is handled for you by the locked stub code in your editor. The first line contains an integer, `T`, the number of test cases. Each of the `T` subsequent lines describes a test case in 2 space-separated integers that denote `n` and `p`, respectively.

Constraints

- No Test Case will result in overflow for correctly written code.

Output Format

Output to stdout is handled for you by the locked stub code in your editor. There are `T` lines of output, where each line contains the result of n^p as calculated by your Calculator class' power method.

Sample Input

```
4
3 5
2 4
-1 -2
-1 3
```

Sample Output

```
243
16
n and p should be non-negative
n and p should be non-negative
```

Change Theme

Language

C#

Exit Full Screen View

```
1 using System;
2
3
4 //Write your code here
5 class Calculator{
6     public int power(int n, int p){
7         if(n < 0 || p < 0){
8             throw new Exception("n and p should be non-negative");
9         }
10        return (int)Math.Pow(n,p);
11    }
12 }
13
14 class Solution{
15     static void Main(String[] args){
16         Calculator myCalculator=new Calculator();
17         int T=Int32.Parse(Console.ReadLine());
18         while(T-->0){
19             string[] num = Console.ReadLine().Split();
20             int n = int.Parse(num[0]);
21             int p = int.Parse(num[1]);
22             try{
23                 int ans=myCalculator.power(n,p);
24                 Console.WriteLine(ans);
25             }
26             catch(Exception e){
27                 Console.WriteLine(e.Message);
28             }
29         }
30     }
31 }
32
33
34 }
```

Line: 13 Col: 16

Upload Codes as File

Test against custom input

Run Code

Submit Code

Submissions

Leaderboard

Discussions

Editorial

Tutorial

HackerRank

Prepare > Tutorials > 30 Days of Code > Day 17: More Exceptions

Objective

Yesterday's challenge taught you to manage exceptional situations by using try and catch blocks. In today's challenge, you will practice throwing and propagating an exception. Check out the [Tutorial](#) tab for learning materials and an instructional video.

Task

Write a Calculator class with a single method: `int power(int n, int p)`. The power method takes two integers, `n` and `p`, as parameters and returns the integer result of n^p . If either `n` or `p` is negative, then the method must throw an exception with the message: `n and p should be non-negative`.

Note: Do not use an access modifier (e.g.: `public`) in the declaration for your Calculator class.

Input Format

Input from stdin is handled for you by the locked stub code in your editor. The first line contains an integer, `T`, the number of test cases. Each of the `T` subsequent lines describes a test case in 2 space-separated integers that denote `n` and `p`, respectively.

Constraints

- No Test Case will result in overflow for correctly written code.

Output Format

Output to stdout is handled for you by the locked stub code in your editor. There are `T` lines of output, where each line contains the result of n^p as calculated by your Calculator class' power method.

Sample Input

```
4
3 5
2 4
-1 -2
-1 3
```

Sample Output

```
243
16
n and p should be non-negative
n and p should be non-negative
```

Line: 13 Col: 16

Upload Codes as File

Test against custom input

Run Code

Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#) [Next Challenge](#)

Test case 0

Test case 1

Input (stdin)

```
4
3 5
2 4
-1 -2
-1 3
```

Download

Expected Output

```
243
16
n and p should be non-negative
n and p should be non-negative
```

Download